



Dynamic Programming(DP 다이나믹 프로그래밍 알고리즘)

<https://youtu.be/3N33ONi380Y?feature=shared>

동적 계획법 (Dynamic Programming - DP)

💡 핵심 개념

DP 이전단계에서 구했던 최적의 값과, 현재상황에서 구한 값을 비교하여 더 최적의 값을 찾는 알고리즘이다.

동적 계획법의 특징

- 중복 부분 문제: 같은 문제를 반복적으로 해결
- 최적 부분 구조: 작은 문제의 해가 큰 문제의 해에 포함
- 메모이제이션: 한 번 계산한 결과를 저장해서 재사용

🔍 그래프 탐색 vs 동적 계획법

그래프 탐색 (BFS/DFS)

위 문제를 해결하기 위한 대표적인 알고리즘 세가지를 고려해 볼 수 있다.

1. 모든 경우의 수를 다 해볼 것인가? 브루트포스 (DFS, BFS)

DP의 장점

2. 최적의 해를 항상 그리기에 찾는다. 그리디 알고리즘

과거 선택에대한 고려가 전혀 없어 미래를 선택하는 부분도 전혀 고려를 하지 않고 현재의 한때 선택한다.

3. DP 이전단계에서 구했던 최적의 값과, 현재상황에서 구한 값을 비교하여 더 최적의 값을 찾는 알고리즘이다.

```
// factorial dp, 재귀함수 비교 예시
#include <iostream>
#include <vector>

int factorial(int n)
{
    if (n <= 1) return 1;
    return n * factorial(n - 1);
}

int factorial_dp(int n)
{
    if (n < 0)
        return -1;
```

```

std::vector<int> dp(n + 1);
dp[0] = 1;

for (size_t i = 1; i < n + 1; i++)
    dp[i] = dp[i - 1] * i;

return dp[n];
}

int main()
{
    int fact = factorial(5);
    fact = factorial_dp(5);

    return 0;
}

```

🪙 동전 문제 (Coin Change Problem)

문제 상황

또 다른 예로 앞서 풀어봤던 문제죄에 거스름돈 문제가 있다.

화폐단위: 500원, 100원, 50원, 10원
손님이 받을 개수: 2개, 2개, 1개, 1개

1260원을 만드는 경우 이것 또한 그리디 알고리즘의 하나로 볼 수 있다.

그리디 알고리즘의 한계

3,4,5원의 동전을 가지고 15원을 만들어보라고 가정해보자.

그리디로 해결하면:

- 5원 × 3개 = 15원 
- 하지만 다른 조합도 가능: 3원 × 5개 = 15원

📊 DP 테이블로 문제 해결

기본 DP 문제 설정

위문제를 DP로 풀어보자

정확하고 대표적인 기본 DP 문제이다. dp 테이블을 만들어서 문제를 해결하면 된다.

만들어야 할 테이블의 크기를 정해준다. 금액보다 +1 한개 크게 만들어주면 된다.

3원 동전만 사용하는 경우

3원의 동전만 가지고 15원의 거스름을 가지려 좋지 않는 경우의수를 테이블을 만들어보자

금액	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
3원	1	0	0	0	1	0	0	1	0	0	1	0	0	1	0	0	1

이번에는 3,4원의 동전을 가지고 만들 수 있는 경우의 수를 생각해보자

금액	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	
3원	1	0	0	0	1	0	0	1	0	0	1	0	0	1	0	0	1
3,4원	1	0	0	0	1	1	0	1	1	1	1	1	1	2	1	1	2

금액	0	1	2	3	4	5	6
3	1	0	0	1	0	0	1

금액	0	1	2	3	4	5	6
3,4	1	0	0	1	1	0	1
3,4,5	1	0	0	1	1	1	1

거스름돈이 4원이 만들어질때까지는 3원까지의 조합과 동일하다.

DP 알고리즘 단계별 분석

DP 점화식 도출

논리색이 현재 동전을 추가하기전 거슬러 줄 수 있는 경우의수 + 현재 가장 큰 동전을 하나 먼저 거슬러주고 남은 돈을 거슬러 줄 수 있는지에 대한 정답이다.

0+0 은 0 이기 때문에 (3,4) 조합으로 5원을 거슬러 줄 수 있다

$dp[index] += dp[index - \text{현재 동전}]$

4원 동전 추가 후 변화

4원 부터는 달라진다.

금액	0	1	2	3	4
3원	1	0	0	1	0
3,4원	1	0	0	1	1

5원의 경우에는 3,4원으로 만들 수 있는 경우의수가 없다.

그렇다면 3,4원으로 5원을 거슬러 줄 경우에는 4원을 내주고 1원을 거슬러 줄 수 있는 경우의 조합이 있을 것이 볼 수 있다.

금액	0	1	2	3	4	5
3원	1	0	0	1	0	0
3,4원	1	0	0	1	1	0
3,4,5	1	0	0	1	1	1

DP 구현 코드

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <climits>

using namespace std;

int main()
{
    // 동전 집합과 목표 금액
    vector<int> coins = {3, 4, 5};
    int amount = 15;

    // DP 테이블 초기화
    vector<int> dp(amount + 1, 0);
    dp[0] = 1; // 0원을 만드는 방법은 1가지 (아무것도 선택하지 않기)

    // 각 동전에 대해
    for (int coin : coins)
    {
        // 해당 동전부터 목표 금액까지
        for (int i = coin; i <= amount; i++)
        {
```

```

        dp[i] += dp[i - coin]; // 점화식
    }
}

// 결과 출력
if (dp[amount] == 0)
{
    cout << "해당 금액을 만들 수 없습니다." << endl;
}
else
{
    cout << amount << "원을 만들기 위한 최소 동전 개수: " << dp[amount] << endl;
}

return 0;
}

```

DP 실행 과정 시뮬레이션

3원 동전 처리

```

coin = 3원 때:
i=3: dp[3] += dp[0] = 0 + 1 = 1
i=6: dp[6] += dp[3] = 0 + 1 = 1
i=9: dp[9] += dp[6] = 0 + 1 = 1
...

```

4원 동전 추가

```

coin = 4원 때:
i=4: dp[4] += dp[0] = 0 + 1 = 1
i=7: dp[7] += dp[3] = 0 + 1 = 1
i=8: dp[8] += dp[4] = 0 + 1 = 1
...

```

5원 동전 추가

```

coin = 5원 때:
i=5: dp[5] += dp[0] = 0 + 1 = 1
i=8: dp[8] += dp[3] = 1 + 1 = 2 (3+5 또는 4+4)
i=10: dp[10] += dp[5] = 1 + 1 = 2
...

```

DP 테이블 완성 과정

최종 DP 테이블

결과적으로 이것들은 패턴이 보이기 시작한다.

금액	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
3원	1	0	0	1	0	0	1	0	0	1	0	0	1	0	0	1
3,4원	1	0	0	1	1	0	1	1	1	1	1	1	2	1	1	2

7원의 경우 (현재 동전) 추가되지전 거슬러 줄 수 있는 경우의수 0) + (4원 거슬러주고 남은 3원 거슬러 줄 수 있는 경우의수 1) = 1이 된다.

DP의 핵심 원리

1. 상태 정의

```
dp[i] = i원을 만들 수 있는 경우의 수
```

2. 초기값 설정

```
dp[0] = 1; // 0원을 만드는 방법은 1가지
```

3. 점화식

```
dp[i] += dp[i - coin]; // 현재 동전을 사용해서 만드는 경우
```

4. 계산 순서

- 동전별로 순차 처리
- 각 동전에 대해 해당 동전 값부터 목표까지 계산

다양한 DP 문제 유형

1. 동전 교환 문제 변형들

- 최소 동전 개수: `dp[i] = min(dp[i], dp[i-coin] + 1)`
- 경우의 수: `dp[i] += dp[i-coin]`
- 가능 여부: `dp[i] = dp[i] || dp[i-coin]`

2. 배낭 문제 (Knapsack)

```
dp[i][w] = max(dp[i-1][w], dp[i-1][w-weight[i]] + value[i]);
```

3. 피보나치 수열

```
dp[i] = dp[i-1] + dp[i-2];
```

DP 설계 체크리스트

DP 적용 가능 조건

1. 최적 부분 구조: 작은 문제의 최적해가 큰 문제의 최적해에 포함
2. 중복 부분 문제: 같은 하위 문제가 여러 번 등장
3. 무후효성: 현재 상태가 미래에만 영향, 과거와 독립

DP 구현 단계

1. 상태 정의: `dp[i]` 가 무엇을 의미하는지 명확히
2. 점화식 도출: 상태 간의 관계식 설정
3. 기저 조건: 초기값 설정
4. 계산 순서: Bottom-up 또는 Top-down

연습 문제

1. 계단 오르기: 1칸 또는 2칸씩 올라갈 수 있을 때 n번째 계단에 도달하는 방법의 수
2. 최장 증가 부분 수열: 배열에서 증가하는 부분 수열의 최대 길이
3. 편집 거리: 두 문자열을 같게 만들기 위한 최소 편집 횟수
4. 0-1 배낭 문제: 용량 제한이 있는 배낭에 최대 가치로 물건 담기

피보나치 수열 DP 분석

문제 정의

피보나치 수열: $F(n) = F(n-1) + F(n-2)$, $F(0) = 0$, $F(1) = 1$

수열: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, ...
 $F(10) = 55$ 를 구해보자

일반 재귀 vs DP 비교

일반 재귀 (비효율적)

```
int fibonacci_recursive(int n)
{
    if (n <= 1) return n;
    return fibonacci_recursive(n-1) + fibonacci_recursive(n-2);
}
```

문제점: $F(5)$ 를 계산할 때

```
      F(5)
     /  \
    F(4)  F(3)
   / \  / \
  F(3) F(2) F(2) F(1)
... (중복 계산 발생)
```

시간복잡도: $O(2^n)$ - 매우 비효율적!

DP 해결 (효율적)

```
int fibonacci_dp(int n)
{
    vector<int> dp(n + 1);
    dp[0] = 0; // 기저조건
    dp[1] = 1; // 기저조건

    for (int i = 2; i <= n; i++)
    {
        dp[i] = dp[i-1] + dp[i-2]; // 점화식
    }

    return dp[n];
}
```

시간복잡도: $O(n)$ - 효율적!

DP 테이블 구성 과정

$F(10)$ 을 구하는 과정:

i	0	1	2	3	4	5	6	7	8	9	10
dp[i]	0	1	1	2	3	5	8	13	21	34	55

단계별 계산:

```
dp[2] = dp[1] + dp[0] = 1 + 0 = 1
dp[3] = dp[2] + dp[1] = 1 + 1 = 2
dp[4] = dp[3] + dp[2] = 2 + 1 = 3
dp[5] = dp[4] + dp[3] = 3 + 2 = 5
```

```
...
dp[10] = dp[9] + dp[8] = 34 + 21 = 55
```

완전한 피보나치 DP 구현

```
#include <iostream>
#include <vector>

using namespace std;

int main()
{
    int n = 10; // F(10) 구하기

    // DP 테이블 초기화
    vector<int> dp(n + 1);

    // 기저 조건
    dp[0] = 0;
    dp[1] = 1;

    cout << "피보나치 수열 계산 과정:" << endl;
    cout << "F(0) = " << dp[0] << endl;
    cout << "F(1) = " << dp[1] << endl;

    // Bottom-up 방식으로 계산
    for (int i = 2; i <= n; i++)
    {
        dp[i] = dp[i-1] + dp[i-2];
        cout << "F(" << i << ") = F(" << (i-1) << ") + F(" << (i-2)
            << ") = " << dp[i-1] << " + " << dp[i-2] << " = " << dp[i] << endl;
    }

    cout << "\n최종 결과: F(" << n << ") = " << dp[n] << endl;

    return 0;
}
```

배낭 문제 완전 정복 - 게임 아이템 수집하기

문제 상황: RPG 게임의 던전 탐험

배경 스토리

용사가 던전을 탐험하며 보스를 처치했습니다! 보상으로 귀중한 아이템들을 발견했지만, 인벤토리 용량이 한정되어 있어 모든 아이템을 가져갈 수 없습니다.

인벤토리 용량: 5칸

발견한 아이템들:

마법반지	2칸	가치 3	💰💰
× 신검	3칸	가치 4	💰💰💰
🛡️ 용방패	4칸	가치 5	💰💰💰💰
👑 왕의왕관	5칸	가치 6	💰💰💰💰💰

? 문제: 인벤토리에 담을 수 있는 최대 가치는?

DP 테이블 설계 및 점화식

상태 정의

$dp[i][w]$ = i 번째 아이템까지 고려했을 때,
용량 w 에서 얻을 수 있는 최대 가치

점화식

```
dp[i][w] = max(  
    dp[i-1][w], // 현재 아이템을 선택하지 않는 경우  
    dp[i-1][w-weight[i]] + value[i] // 현재 아이템을 선택하는 경우  
    // 현재무게를 뺀 나머지 무게의 가치 + 현재무게의 가치  
)
```

경계 조건

```
if (weight[i] > w) {  
    dp[i][w] = dp[i-1][w]; // 용량 초과 시 선택 불가  
}
```

단계별 DP 테이블 구성

아이템 정보 정리

아이템 1:  마법반지 (2칸, 가치 3)
아이템 2:  신검 (3칸, 가치 4)
아이템 3:  용방패 (4칸, 가치 5)
아이템 4:  왕의왕관 (5칸, 가치 6)

초기화

용량 →	0	1	2	3	4	5
아이템 ↓						
0개 (없음)	0	0	0	0	0	0

1단계: 마법반지 (2칸, 가치 3) 추가

 각 용량별 계산:
- 용량 0~1: 마법반지(2칸)이 안들어감 → 0
- 용량 2~5: 마법반지 들어감 → 가치 3

용량 →	0	1	2	3	4	5
아이템 ↓						
0개 (없음)	0	0	0	0	0	0
1개 (반지)	0	0	3	3	3	3

2단계: 신검 (3칸, 가치 4) 추가

 핵심 계산 (용량 5):
 $dp[2][5] = \max(
 dp[1][5], // 신검 안가져가기 = 3
 dp[1][5-3] + 4 // 신검 가져가기 = $dp[1][2] + 4 = 3 + 4 = 7$
)
→ $\max(3, 7) = 7$ 선택! (반지 + 신검)$

용량 →	0	1	2	3	4	5
아이템 ↓						
0개 (없음)	0	0	0	0	0	0

1개 (반지)	0	0	3	3	3	3
2개 (신검)	0	0	3	4	4	7

3단계: 용방패 (4칸, 가치 5) 추가

```

💡 핵심 계산 (용량 5):
dp[3][5] = max(
    dp[2][5],          // 용방패 안가져가기 = 7
    dp[2][5-4] + 5     // 용방패 가져가기 = dp[2][1] + 5 = 0 + 5 = 5
)
→ max(7, 5) = 7 선택! (반지 + 신검이 더 좋음)

용량 → 0  1  2  3  4  5
아이템 ↓
0개 (없음)    0  0  0  0  0  0
1개 (반지)    0  0  3  3  3  3
2개 (신검)    0  0  3  4  4  7
3개 (용방패)  0  0  3  4  5  7

```

4단계: 왕의왕관 (5칸, 가치 6) 추가

```

💡 핵심 계산 (용량 5):
dp[4][5] = max(
    dp[3][5],          // 왕관 안가져가기 = 7
    dp[3][5-5] + 6     // 왕관 가져가기 = dp[3][0] + 6 = 0 + 6 = 6
)
→ max(7, 6) = 7 선택! (반지 + 신검 조합이 최고)

용량 → 0  1  2  3  4  5
아이템 ↓
0개 (없음)    0  0  0  0  0  0
1개 (반지)    0  0  3  3  3  3
2개 (신검)    0  0  3  4  4  7
3개 (용방패)  0  0  3  4  5  7
4개 (왕관)    0  0  3  4  5  7 ← 최종!

```

💻 완전한 C++ 구현

```

#include <iostream>
#include <vector>
#include <algorithm>
#include <string>

using namespace std;

int main()
{
    // 게임 설정
    int capacity = 5;

    vector<string> items = { "🧙마법반지", "⚔️신검", "🛡️용방패", "👑왕의왕관" };
    vector<int> weights = { 2, 3, 4, 5 };
    vector<int> values = { 3, 4, 5, 6 };
    int n = items.size();

    // DP 테이블 초기화
    vector<vector<int>> dp(n + 1, vector<int>(capacity + 1, 0));

    cout << "🎮 RPG 게임 - 던전 아이템 수집하기" << endl;
    cout << "🎒 인벤토리 용량: " << capacity << "칸\n" << endl;

    // 아이템 정보 출력
    cout << "💎 발견한 아이템들:" << endl;

```

```

for (int i = 0; i < n; i++)
{
    cout << " " << (i + 1) << ". " << items[i]
        << " (용량: " << weights[i] << "칸, 가치: " << values[i] << ")" << endl;
}
cout << endl;

// DP 계산
for (int i = 1; i <= n; i++) {
    cout << "📄 " << i << "단계: " << items[i - 1] << " 검토 중..." << endl;

    for (int w = 0; w <= capacity; w++)
    {
        if (weights[i - 1] > w)
        {
            // 현재 아이템이 용량을 초과하는 경우
            dp[i][w] = dp[i - 1][w];
        }
        else
        {
            // 선택하지 않는 경우와 선택하는 경우 중 최대값
            int not_take = dp[i - 1][w];
            int take = dp[i - 1][w - weights[i - 1]] + values[i - 1];
            dp[i][w] = max(not_take, take);

            // 최대 용량에서의 계산 과정 출력
            if (w == capacity)
            {
                cout << " 용량 " << w << "칸: max(" << not_take
                    << ", " << take << ") = " << dp[i][w] << endl;
            }
        }
    }
    cout << endl;
}

// 최종 DP 테이블 출력
cout << "📊 최종 DP 테이블:" << endl;
cout << " ";
for (int w = 0; w <= capacity; w++)
{
    cout << "c" << w << " ";
}
cout << endl;

for (int i = 0; i <= n; i++)
{
    if (i == 0)
    {
        cout << "없음: ";
    }
    else
    {
        cout << i << "개: ";
    }
    for (int w = 0; w <= capacity; w++)
    {
        cout << dp[i][w] << " ";
    }
}

```

```

    }
    cout << endl;
}

cout << "\n🏆 최대 가치: " << dp[n][capacity] << endl;

// 선택된 아이템 역추적
cout << "\n📦 선택된 아이템들:" << endl;
vector<bool> selected(n, false);
int w = capacity;

for (int i = n; i > 0 && w > 0; i--)
{
    if (dp[i][w] != dp[i - 1][w])
    {
        selected[i - 1] = true;
        cout << "   ✓ " << items[i - 1]
            << " (용량: " << weights[i - 1] << "칸, 가치: " << values[i - 1] << ")" << endl;
        w -= weights[i - 1];
    }
    else
    {
        cout << "   ✗ " << items[i - 1] << " (선택하지 않음)" << endl;
    }
}

// 최종 요약
int total_weight = 0, total_value = 0;
for (int i = 0; i < n; i++)
{
    if (selected[i])
    {
        total_weight += weights[i];
        total_value += values[i];
    }
}

cout << "\n📋 최종 요약:" << endl;
cout << "   총 사용 용량: " << total_weight << "/" << capacity << "칸" << endl;
cout << "   총 가치: " << total_value << endl;
cout << "   남은 용량: " << (capacity - total_weight) << "칸" << endl;

return 0;
}

```

🏆 결과 분석 및 역추적

최적해: 가치 7

📦 선택된 아이템 조합:
 ✓ 🧙 마법반지 (2칸, 가치 3)
 ✓ ✕ 신검 (3칸, 가치 4)
 ✗ 🛡 용방패 (선택하지 않음)
 ✗ 🏰 왕의왕관 (선택하지 않음)

총 사용 용량: $2 + 3 = 5$ 칸 (꼭 참!)
 총 가치: $3 + 4 = 7$

역추적 과정

1. $dp[4][5] = 7$, $dp[3][5] = 7$ → 왕관 선택 안함 ❌
2. $dp[3][5] = 7$, $dp[2][5] = 7$ → 용방패 선택 안함 ❌
3. $dp[2][5] = 7$, $dp[1][5] = 3$ → 신검 선택! ✅ (남은 용량: $5-3=2$)
4. $dp[1][2] = 3$, $dp[0][2] = 0$ → 마법반지 선택! ✅

왜 이 조합이 최적일까?

다른 가능한 조합들:

- 👑 왕관만: 5칸, 가치 6
- 🛡️ 용방패만: 4칸, 가치 5
- 📖 반지 + ✂️ 신검: 5칸, 가치 7 ← 최고!

작은 아이템들의 조합이 큰 아이템 하나보다 더 효율적!

💡 핵심 개념 정리

1. DP의 핵심 아이디어

- **최적 부분 구조**: 작은 문제의 최적해로 큰 문제의 최적해 구성
- **중복 부분 문제**: 같은 계산을 여러 번 하지 않도록 결과 저장
- **점진적 구성**: 아이템을 하나씩 추가하며 최적해 갱신

2. 선택의 기준

각 단계에서 두 가지 선택:

1. 현재 아이템 선택 안함 → 이전 상태 그대로
2. 현재 아이템 선택함 → 남은 공간의 최적해 + 현재 아이템 가치

3. 시간/공간 복잡도

- **시간 복잡도**: $O(n \times W) = O(4 \times 5) = O(20)$
- **공간 복잡도**: $O(n \times W) = O(20)$

4. 실전 팁

- **작은 아이템 조합 vs 큰 아이템**: 효율성 비교 중요
- **역추적**: 최적해가 어떤 선택들로 구성되었는지 파악
- **경계 조건**: 용량 초과 시 처리 주의

🎮 간단한 4개 아이템으로 배우는 완벽한 배낭 문제! 🏆

📊 세 가지 DP 문제 비교

구분	동전 문제	피보나치	배낭 문제
DP 차원	1차원	1차원	2차원
상태 정의	$dp[i] = i$ 원을 만드는 경우의 수	$dp[i] = i$ 번째 피보나치 수	$dp[i][w] = i$ 개 물건, 용량 w 에서 최대 가치
점화식	$dp[i] += dp[i-coin]$	$dp[i] = dp[i-1] + dp[i-2]$	$dp[i][w] = \max(dp[i-1][w], dp[i-1][w-weight] + value)$
시간복잡도	$O(\text{amount} \times \text{coins})$	$O(n)$	$O(n \times W)$
공간복잡도	$O(\text{amount})$	$O(n)$	$O(n \times W)$

🎯 DP 마스터 체크리스트

✅ 문제 분석 단계

1. 중복 부분 문제가 있는가?

- 같은 하위 문제가 여러 번 계산되는지 확인

2. 최적 부분 구조를 만족하는가?

- 작은 문제의 최적해가 큰 문제의 최적해에 포함되는지 확인

3. 무후효성을 만족하는가?

- 현재 상태가 과거 경로와 무관하게 결정되는지 확인

구현 전략

1. Top-down (재귀 + 메모이제이션)

```
int dp[MAX];
int solve(int n)
{
    if (dp[n] != -1) return dp[n]; // 이미 계산됨
    return dp[n] = /* 점화식 */;
}
```

2. Bottom-up (반복문)

```
vector<int> dp(n + 1);
dp[0] = /* 초기값 */;
for (int i = 1; i <= n; i++)
{
    dp[i] = /* 점화식 */;
}
```

최적화 기법

1. 공간 최적화: 필요한 이전 상태만 저장
2. 상태 압축: 2차원을 1차원으로 변환
3. 계산 순서 최적화: 불필요한 계산 생략

실전 DP 문제 해결 전략

문제 접근 순서

1. 브루트포스 솔루션 먼저 생각

- 모든 경우를 다 해보는 재귀 함수 작성
- 시간복잡도가 너무 크다면 DP 고려

2. 상태와 점화식 정의

- `dp[i]` 가 무엇을 나타내는지 명확히 정의
- 상태 간의 관계식 도출

3. 기저 조건 설정

- 가장 작은 문제의 답을 직접 설정
- 경계 조건 처리

4. 계산 순서 결정

- 의존성을 고려한 계산 순서
- Bottom-up vs Top-down 선택

디버깅 팁

1. 작은 케이스로 손 계산: 점화식이 맞는지 검증

2. DP 테이블 출력: 중간 과정 확인
3. 경계 조건 체크: 배열 범위 초과 방지
4. 초기값 확인: 기저 조건이 올바른지 검토

🎯 핵심 정리

1. DP = 과거의 최적해 + 현재 선택 = 새로운 최적해
2. 테이블 크기: 보통 문제 범위 + 1
3. 점화식이 핵심: 상태 간의 관계를 정확히 파악
4. 초기값 중요: 기저 조건을 올바르게 설정

↩️ 현실적인 DP 학습 과정:

1 초보자의 전형적인 시행착오:

문제를 본다 → "어... 뭔가 복잡하네?"
→ 그리디로 시도 → 반례 발견 🤯
→ 브루트포스로 시도 → 시간초과 🤯
→ "이게 뭘 문제지?" → 답 확인 → "아, DP구나..."

2 점진적 감각 발달:

5-10문제 후: "어? 이거 아까 본 패턴 같은데?"
20-30문제 후: "이런 키워드 나오면 DP일 확률 높아"
50문제 후: "문제 읽자마자 DP 냄새가 난다"
100문제 후: "이건 1차원 DP, 저건 2차원 DP"

3 실제 도움되는 학습법:

💡 의도적으로 실패해보기

```
// 1. 일부러 재귀로 먼저 풀어보기
int slowFib(int n)
{
    if (n <= 1) return n;
    return slowFib(n-1) + slowFib(n-2);
}

// 2. "어? 너무 느리네" 경험하기
// 3. 같은 계산 반복 발견하기
// 4. "아하! 이걸 저장하면 되겠다" 깨달음
```

📚 문제 유형별 외우기

그냥 외워버리는 것도 방법:

- "경우의 수" → DP 의심
- "최소 개수" → DP 의심
- "배낭, 동전, 계단" → 무조건 DP
- "피보나치 같은 점화식" → DP

정말 솔직히 말하면, 처음 20-30문제는 답 보고 "아, 이런 게 DP구나" 하면서 패턴을 익히는 게 가장 효율적입니다!

무작정 고민하다가 시간만 낭비하는 것보다는, 답을 보고 "왜 DP인지" 이해하는 게 더 도움이 돼요. 🚀✨

“강의는 많은데, 왜 나는 아직도 코드를 못 짤까?”

혼자 공부하다 보면 누구나 이런 고민을 하게 됩니다.

- 강의는 다 들었지만 막상 **손이 안 움직이고**,
- 복습을 하려 해도 **무엇을 다시 봐야 할지 모르겠고**,
- 질문할 곳도 없고,
- 유튜브는 결국 **정답을 따라 치는 것밖에 안 되는 것 같고**.

문제는 ‘**연습**’이 **빠졌기 때문**입니다.

단순히 강의를 듣는 것만으로는 실력이 늘지 않습니다.

실제 문제를 풀고, 고민하고, 직접 구현해보는 시간이 반드시 필요합니다.

그래서, 암암코딩 코칭은 다릅니다.

그냥 가르치지 않습니다.

스스로 설계하고, 코딩할 수 있게 만듭니다.

암암코딩 코칭에서는 단순한 예제가 아닌,

스스로 문제를 분석하고 구현해야 하는 **연습문제**를 제공합니다.

이 연습문제들은 다음과 같은 역량을 키우기 위해 설계되어 있습니다:

- 문제를 스스로 **쫓개고 설계하는 힘**
- 다양한 조건을 만족시키는 **실제 구현 능력**
- 기능 단위가 아닌, **프로그램 단위로 사고하는 습관**
- 마침내 **자신의 힘으로 코드를 끝까지 작성하는 경험**

지금 필요한 건 더 많은 강의가 아닙니다.

코드를 스스로 완성해 나가는 **훈련**,

그것이 지금 실력을 끌어올릴 가장 현실적인 방법입니다.

👉 자세한 안내 보기: [프리미엄 코칭 안내 바로가기](#)

👉 또는 카카오톡 상담방: [암암코딩 상담방](#)

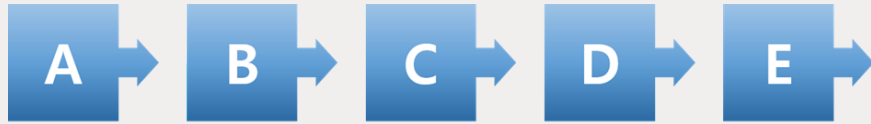
▼ oopy:hide

연습문제



문제 1번

링크드리스트 복습입니다.



Memory Pool 을 사용하여 위와 같이 링크드리스트를 만들어 보세요.

아래 코드를 완벽히 이해한 후, 소스코드를 베끼지 않고, 직접 코딩해서 이 문제를 풀어보세요!

[출력]

For문으로 A ~ E 까지 출력해주세요.

```
struct Node
{
    char ch;
    Node* next;
};
Node buf[100];
int bufCnt;
Node* head;
Node* myAlloc(int ch, Node * next)
{
    buf[bufCnt].ch = ch;
    buf[bufCnt].next = next;
    return &buf[bufCnt++];
}

void addNode(char ch, Node * next)
{
    head = myAlloc(ch, head);
}

int main()
{
    return 0;
}
```

출력 결과

A B C D E



문제 2번

입력 받은 글자를 링크드리스트에 저장하고 출력 해주세요.

글자들을 역순으로 링크드리스트에 저장하고 for문을 돌려 두 칸씩 뛰어넘으면서 출력하세요.

[예시]

입력 받은 C O D I N G 를 역순으로 저장하면 아래처럼 링크드리스트를 구성이 됩니다.

이제 맨 앞부터 값을 수 있는 징검다리인 돌 들을 두 칸씩 뛰어넘으면서 출력하면 됩니다.

이때 출력결과는 G I O N M 입니다.

[입력]

총 9글자를 입력 받으세요.

[출력]

입력 받은 글자를 링크드리스트에 한 글자씩 역순으로 저장하고, 맨 앞부터 두 칸씩 건너뛰어 출력하세요.

입력 예제

C O D I N G

출력 결과

G I O



문제 3번

음계를 입력 받고 링크드리스트에 저장 후, 출력 해주세요.

Memory Pool 이용하여 링크드리스트를 만들고, 음계 정보를 순차적으로 저장 하세요.

그리고 링크드리스트를 그대로 출력 해주세요.

- 정방향으로 링크드리스트가 만들어 진다는 것을 유의하세요.



[입력]

음계의 개수 N을 입력 받으세요.

그 다음 줄의 각 음계의 값을 입력 받으세요.

[출력]

저장된 링크드리스트를 출력 해주세요.

입력 예제

6

5 3 4 1 2 7

출력 결과

5 3 4 1 2 7



문제 4번

8 마리의 공룡 화석이 발견되었고, 이름과 나이는 아래와 같습니다.

Sour	Dav	Nica	Timer	Pico	Topisl	Whab	Hap
							
↑	↑	↑	↑	↑	↑	↑	↑
1,000,000,005	1,000,000,002	1,000,000,003	1,000,000,006	1,000,000,015	1,000,000,022	1,000,000,013	1,000,000,009

공룡들의 나이를 DB에 저장하고 공룡들을 찾을 수 있는 시스템을 만드려고 합니다.

DB 사용자들이 공룡의 나이를 입력하면 DB에서 해당 나이에 맞는 공룡의 이름을 검색 합니다.

- 공룡들 나이 중 같은 나이는 없습니다.

[입력]

공룡의 나이를 입력 받습니다.

[출력]

나이에 해당하는 공룡의 이름을 출력합니다.

[힌트1]

공룡들의 나이가 1억이 넘기에 배열에 바로 담을 수 없습니다.

Hash Function을 써서 bucket에 값을 담고, 바로 찾으시면 됩니다.

[힌트2]

입력 예제

10000000005

출력 결과

Sour



문제 5번

ID/PASSWORD를 입력받으면 암호화를 해서 Server에 전송하곤합니다.
Hash 함수를 이용하여 입력받은 ID를 암호화 후 출력 해 주세요.

- HashFunction : hornor's method 이용

계산은 아스키코드 번호를 그대로 사용하면됩니다.

[예시]

A : 65

B : 66

C : 67

...

JASON

$$J_{x26}^4 + A_{x26}^3 + S_{x26}^2 + O_{x26}^1 + N_{x26}^0$$

[입력]

대문자로 문자를 입력 받으세요.

[출력]

암호화 결과 값을 출력하세요.

입력 예제

JASON

출력 결과

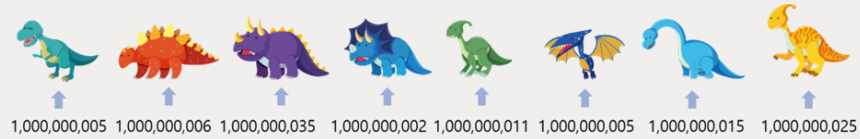
35016904



문제 6번

공룡의 화석 8개가 발견되었습니다.

이 화석으로 공룡의 나이 조사 결과, 대부분 10억살 이상이었고 정확한 나이는 다음과 같습니다.

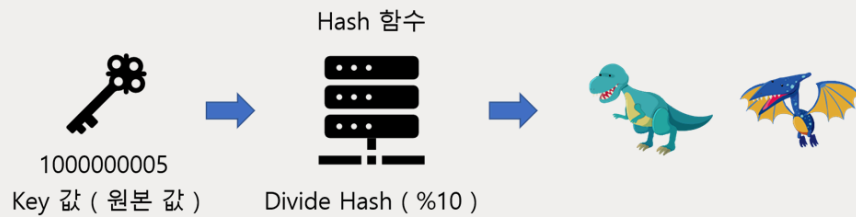


숫자를 입력 받고, 이 숫자와 같은 나이를 가진 공룡이 총 몇마리인지 출력 해주세요.

[힌트1]

숫자가 너무 커서 이 값을 배열의 index로 쓸 수 없기 때문에 Hash Function이 필요합니다.

그런데 Hash Function을 거치면 다른 값들이 같은 index에 들어갈 수 있기 때문에, Chaining이 필요합니다.



[힌트2]

HashTable(Bucket)에 담고, 내가 찾는 target값이 몇 개인지만 세면 됩니다.

[입력]

찾을 숫자를 입력 받으세요.

[출력]

몇마리가 해당 나이와 같은지 마리수를 출력 해주세요.

입력 예제

10000000005

출력 결과

2마리



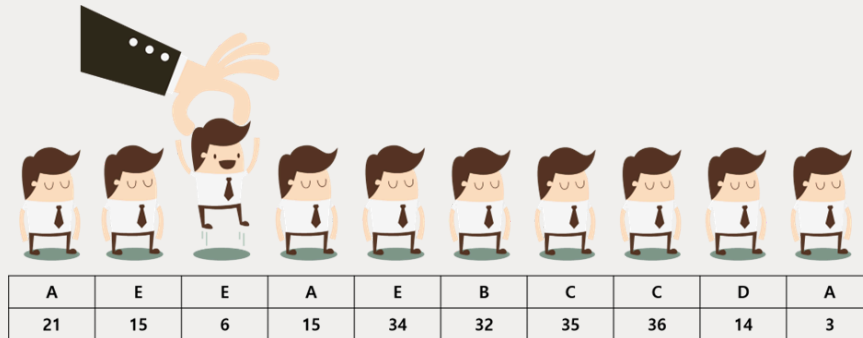
문제 7번

A ~ E 그룹사에서 신입사원들이 모였습니다.

그룹사별 신입사원들의 정보를 Memory Pool + Chaining 을 사용하여 저장해주세요.

그룹사의 이름을 입력 받고 해당 그룹사의 신입사원 ID 들을 모두 출력 해주세요.

- Hash Function 은 사용하지 않아도 됩니다.



A	E	E	A	E	B	C	C	D	A
21	15	6	15	34	32	35	36	14	3

[입력]

출력할 그룹사의 이름을 입력하세요.

[출력]

해당 그룹사의 출력할 ID 는 위 배열의 왼쪽부터 오른쪽 순서대로 출력 해주세요.

입력 예제

A

출력 결과

21 15 3



문제 8번

첩보원 코드 번호를 입력하면, 몇 명을 암살했는지 출력해주는 프로그램을 만들어야합니다.
 한글자로 된 첩보원 코드번호와 그들이 암살에 성공한 적들의 수가 아래 처럼 저장되어 있습니다.
 첩보원이 적에게 당하지 않도록, Hash를 써서 빠르고 정확한 프로그램을 만들어주어야 합니다.
 B를 입력하면 10이 출력되고, Z가 입력되면 16이 출력되는 프로그램을 작성 해 주세요.

[데이터 정보]

다음은 시스템에 미리 저장되어야 할 기초 데이터입니다.

첩보 코드 (Key)	암살 횟수 (Value)
'A'	5
'B'	10
'D'	22
'Z'	16

[입력]

첩보코드를 입력 하세요.

[출력]

첩보원의 암살 성공횟수를 출력하세요.

[힌트1]

```
struct Node
{
    char code;          // 첩보원 코드 (Key)
    int killCount;      // 암살 성공 횟수 (Value)
    Node *next;        // 충돌 시 다음 노드를 가리키는 포인터
};
```

입력 예제

B

출력 결과

10

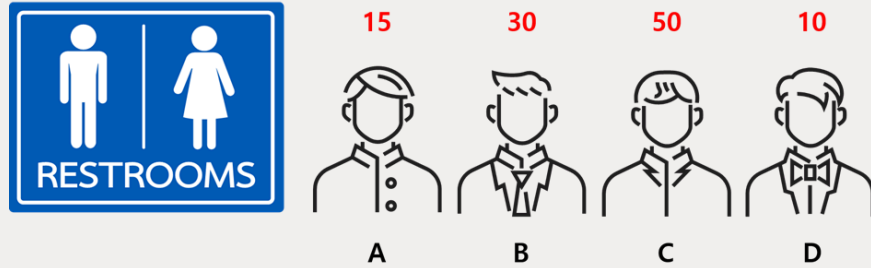


문제 1번

화장실이 하나이기 때문에 화장실을 사용하려면 대기를 해야합니다.

기다리는 사람들이 모두 화장실을 사용한다고 할때, 총 대기시간이 적은 사용 순서를 정해주세요.

예를들어 총 네 명, A ~ D 사람이 화장실을 사용하는 시간이 다음과 같습니다.



한 사람이 화장실을 이용할 때 다른 사람들은 그 사람이 나오길 기다립니다.

사람들의 전체 대기시간이 최소가 되도록 사용순서를 정하고, 최소 대기시간을 출력해주세요.

[입력]

네 사람의 대기 시간을 입력 하세요.

[출력]

화장실을 모든 사람들이 사용완료 했을때 모든 사람이 대기해야하는 총 대기시간 들중, 최소 대기 시간을 출력하세요.

[예시]

정답은 아니지만, A → B → C → D 순서로 화장실에 들어간다고 할 경우

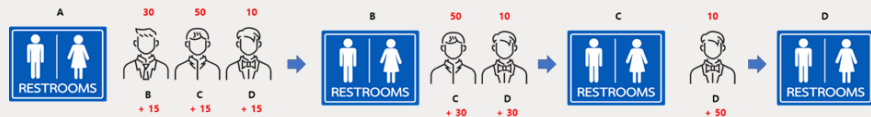
네 사람의 총 대기시간은 155분입니다.

처음 A가 화장실 사용시 : 15 X 3

이후, B가 화장실 사용시 : 30 X 2

이후, C가 화장실 사용시 : 50

마지막으로, D가 화장실을 사용하면 네 명의 각각 대기시간을 합치면 155 입니다.



만약 D > A > B > C 순서대로 화장실을 이용한다면, 총 대기시간은 90분으로 최소 대기시간이 됩니다.

입력 예제

15 30 50 10

출력 결과

90



문제 2번

10원, 50원, 100원, 500원 이렇게 네 종류의 동전이 있습니다.

교환할 동전 금액을 입력받고, 최소의 동전 숫자로 거슬러주려 합니다.

이 문제를 Greedy로 풀어주세요.

만약에 660원을 입력 받았다면,

500원 짜리 1개

100원 짜리 1개

50원 짜리 1개

10원짜리 1개

거슬러 줄 수 있는 최소 동전의 수는 4개 입니다.

[HINT]

10, 50, 100, 500 인 경우는 동전이 서로 배수이기 때문에 Greedy가 성립 됨.

만약 10, 70, 100 인 것처럼 동전이 서로 배수가 아닌 경우 Greedy가 성립 안됨.

입력 예제

660

출력 결과

4



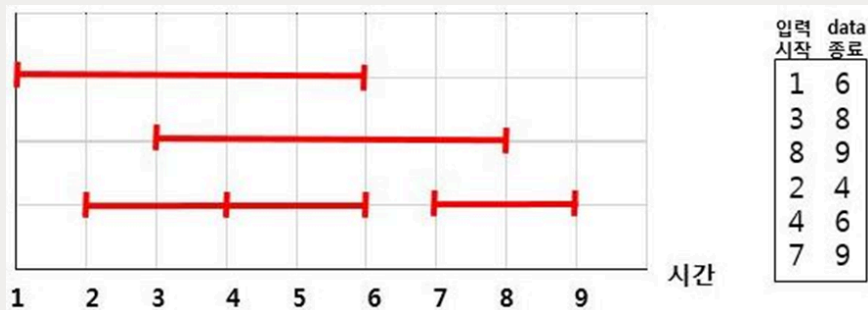
문제 3번

회의실이 하나인 회사가 있습니다.

예약스케줄을 입력받고, 최대 몇번의 회의가 가능한지 출력하세요

만약 6번의 회의가 있고, 아래와 같이 스케줄을 입력받았다면

- > 최대 3번의 회의가 가능합니다.



첫번째 입력숫자는 회의의 횟수입니다.

그 다음 입력 숫자들은 스케줄 시작시간과 종료시간 입니다.

가능한 최대 회의 횟수를 출력하세요

[힌트]

가장 일찍 끝나는 회의 먼저 선택하기

입력 예제

6

1 6

3 8

8 9

2 4

4 6

7 9

출력 결과

3

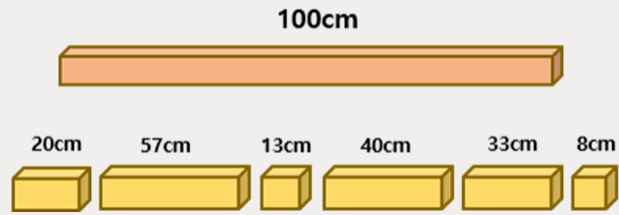


문제 4번

막대 블럭들을 넣을 수 있는 100 cm 크기의 길다란 홈이 있습니다.

n개의 막대 블럭이 있을때 일부 막대블럭을 선택하여

최대 몇개를 넣을 수 있을지 출력하세요



예로 들어,

20cm, 57cm, 13cm, 40cm, 33cm, 8cm 막대 블럭이 있을때 **최대 4개**를 넣을 수 있습니다.

[세부 조건]

최대 입력값은 $1 \leq n \leq 10$ 입니다.

입력 예제

6

20 57 13 40 33 8

출력 결과

4



문제 5번

지게차는 4 x 4 맵에서 (0, 0) 에 있습니다.

그리고 도착지점(3, 3)까지 최단거리로 이동하려고 합니다.

바닥상태가 좋지 않아 땅을 밟을 때 마다, 지게차는 써있는 숫자만큼 손상 되곤 합니다.

최단거리로 도착지점까지 가려고 할 때, **최소 데미지**를 받는 경로를 출력 해 주세요.

만약 아래와 같이 MAP을 입력받는다면

	3	5	1		
↓	1	1	1	→	5
1	50	20	↓	10	
1	50	5	↓	도착	

출력결과는 다음과 같습니다.

0,0

1,0

1,1

1,2

1,3

2,3

3,3

[세부 조건]

1. 맵 사이즈는 4 x 4로 고정입니다.
2. 시작지점(0, 0)과 도착지점(3, 3)의 값은 항상 0 입니다.

입력 예제

0 3 5 1

1 1 1 5

1 50 20 10

1 50 5 0

출력 결과

0,0

1,0

1,1

1,2

1,3

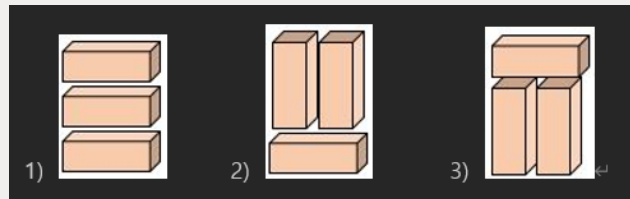
2,3

3,3

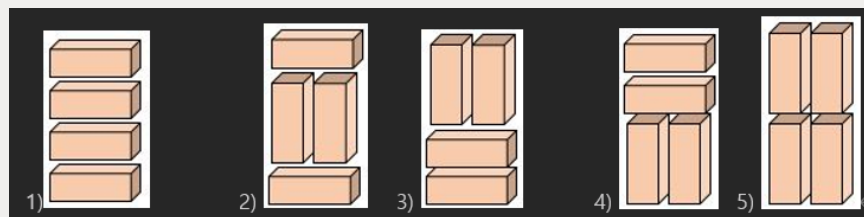
💡 문제 6번



1×2 사이즈의 나무토막이 있습니다.
3단까지 나무를 쌓는 방법은 아래와 같이 총 세가지 입니다.



4단까지 나무를 쌓는 방법은 총 다섯가지 입니다.



n단까지 나무를 쌓으려고 할때 가능한 방법수를 출력해주세요.
(Dynamic Programming 문제 입니다.)

[HINT]

[1]단

- > [0]단에서 가로 1개 쌓기
- > 경우의 수 : 1

[2]단

- > [1]단에서 1개 더 올리기
- + [0]단에서 세로로 쌓기

- > 경우의 수 : 2

[3]단

- > [2]단에서 1개 더 올리기
- > [1]단에서 세로로 쌓기
- > 경우의수 : 3

[4]단

- > [3]단에서 1개 더 올리기
- > [2]단에서 세로로 쌓기
- > 경우의수 : 5

[5]단

- > [4]단에서 1개 더 올리기
- > [3]단에서 세로로 쌓기
- > 경우의 수 : 8

입력 예제

4

출력 결과

5

💡 문제 7번



1		
1	2	2
	3	
3	-10	-5
15	-10	50
15	-10	10
	6	4

7×3 맵을 입력 받으세요.

(0,0) 좌표에 있는 개구리는 로 한 칸 점프 할 수 있습니다. (벽으로 이동 불가)

숫자가 써있는 곳을 밟으면, 그 숫자만큼 점수를 얻습니다.

개구리가 맨 밑까지 내려올 때 개구리가 받을 수 있는 최대 점수는 몇일까요?

(숫자 0은 벽입니다. DP로 풀어주세요.)

입력 예제

```
1 0 0
1 2 2
0 3 0
3 -10 -5
15 -10 50
15 -10 10
0 6 4
```

출력 결과

67



문제 8번

0	1	2	3	4	5	6	7	8	9	10	11	12...
0	2	4	-5	-7	6	20	7	-10	-8	-4	-1	도착

총 12칸으로 된 맵이 있습니다. (0번 index값은 항상 0 입니다.)

블루마일에 사는 부르스 윌리스는 0번에서 시작해서,

(2칸) 또는 (3칸) 또는 (현재 index * 2칸) 으로 점프 할 수 있습니다.

12번 index 이상 지역에 도착 할 때 얻을 수 있는 최대점수를 출력 하세요.

입력 예제

0 2 4 -5 -7 6 20 7 -10 -8 4 -1

출력 결과

21



문제 9번

두문장을 입력받으세요.(최대15글자)

그리고 가장 긴 같은 단어를 찾아주세요.

- 매우 어렵습니다

입력 예제

ABABCGKABABC

BTBCKABABCT

출력 결과

KABABC

HINT

문자열이 겹치는 구간인 KABABC 가 어떻게 쌓이는지 보세요.

	...	K	A
...	0	0	0
K	0	1	0
A	0	0	2
B	0	0	0
A	0	0	1
B	0	0	0
C	0	0	0